Lab 10: Design and Monitor a Multi-Robot Queue

Topic: 5

Objective: LO5 - Optimise robot performance using queues, monitoring and feedback

Alignment: A1, A2, A3, A4, A5, A7, A8, A9 | K3, K9, K13

Suggested time: 80 minutes

Scenario and outcome

Model a dispatcher-worker pattern, calculate queue metrics and decide when to scale, retry or route work for human review.

You will produce: A dispatcher-worker UiPath design, queue simulation workbook and evidence-led optimisation recommendation

Folder contents

- Main.xaml - editable UiPath Studio starter workflow.
- project.json - UiPath project manifest.
- mock-design-and-monitor-a-multi-robot-queue.csv - portable mock source data.
- mock-design-and-monitor-a-multi-robot-queue.xlsx - formatted mock workbook with Data and Rules sheets.
- CHECKLIST.md - completion and evidence checklist.
- ProcessItem.xaml - reusable transaction child workflow.

Preparation

1. Use UiPath Studio Community 2024.10 or later.
2. Extract or clone this entire folder. Keep the data files beside project.json.
3. Open project.json in Studio and allow package restore. Upgrade dependency versions if Studio requests it.
4. Use mock data only. Do not enter passwords, tokens or real personal data.

Procedure

1. Open the queue simulation workbook and identify throughput, success rate, average handling time and retry count.

2. Open Main.xaml and ProcessItem.xaml. Main represents the dispatcher/worker coordinator; ProcessItem has one transaction responsibility.

3. Define queue item data: TransactionID, Priority and input fields. Keep credentials out of item-specific content.

4. In the dispatcher path, read input rows and add one transaction per row with a reference based on TransactionID.

5. In the worker path, get one transaction, invoke ProcessItem.xaml and set Successful, Failed or Business Exception explicitly.

6. Retry only system exceptions and cap attempts. Business exceptions go to human review without blind retry.

7. Calculate metrics from the mock data. Confirm six successful attempts out of eight records, with one business and two distinct system-failure transactions represented.

8. Recommend scale-out only when queue age or throughput evidence supports it. Separate capacity problems from data-quality failures.
9. Capture the dispatcher-worker diagram, metric summary and one optimisation recommendation.

Expected result

Every transaction has a unique reference and final state, retries are bounded, business exceptions are not retried, and the optimisation recommendation cites measured queue evidence.

Troubleshooting

If workers duplicate items, verify unique references and transaction-state updates. If queue age grows while success rate falls, fix the dominant exception before adding capacity.

Evidence and acceptance criteria

Queue metrics workbook, workflow screenshots and a written scale/retry/review recommendation.

The submission is acceptable when the expected result is visible, output files reconcile to the mock input, and the evidence identifies the tested transaction or process.